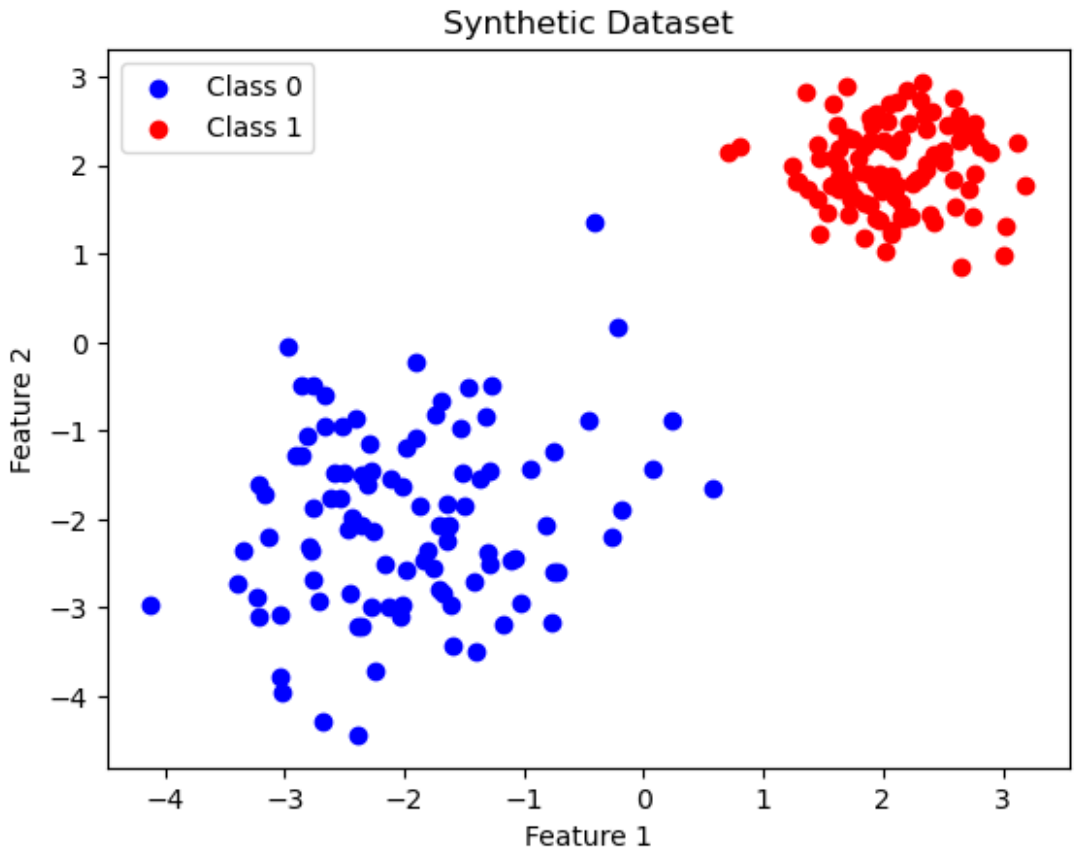
Homework 3: Supervised Learning

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

Exercise 1: Logistic Regression on a Toy 2D Dataset

1. Generate two Gaussian clusters in R^2 and associate them with a class depending on which cluster each point lies on, e.g.:
   - Class 0 centered at $(-2, -2)$ with variance 1,
   - Class 1 centered at $(2, 2)$ with variance 0.5.
2. Plot the dataset in 2D using `plt.scatter` so that each cluster is colored according to its class.

```
n_class_0_samples = 100
n_class_1_samples = 100

class_0 = np.random.normal(0, 1, size=(n_class_0_samples, 2)) +
np.array([-2, -2])
class_1 = np.random.normal(0, 0.5, size=(n_class_1_samples, 2)) +
np.array([2, 2])

plt.scatter(class_0[:, 0], class_0[:, 1], color='blue', label='Class
0')
plt.scatter(class_1[:, 0], class_1[:, 1], color='red', label='Class
1')
plt.title('Synthetic Dataset')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```



1. Implement logistic regression **from scratch** as did during class:

$$f_\Theta(x)=\sigma(\Theta^T x), \ell(\Theta; x, y)=-\left[y \log f_\Theta(x)+(1-y)\log\left(1-f_\Theta(x)\right)\right].$$

```
def sigmoid(t):
    return 1 / (1 + np.exp(-t))

def l(Theta, X, Y):
    Y_hat = sigmoid(X @ Theta)
    Y_hat = np.clip(Y_hat, 1e-15, 1 - 1e-15)  # Avoid log(0)
    return - np.sum(Y * np.log(Y_hat) + (1-Y) * np.log(1 - Y_hat))

def grad_l(Theta, X, Y):
    Y_hat = sigmoid(X @ Theta)
    Y_hat = np.clip(Y_hat, 1e-15, 1 - 1e-15)  # Avoid log(0)
    return X.T @ (Y_hat - Y)

def accuracy(Theta, X, Y):
    preds = sigmoid(X @ Theta) >= 0.5
    return (preds == Y).mean()

def SGD(l, grad_l, X, Y, Theta0, lr=1e-2, batch_size=32, epochs=10):
    epoch_loss = []
```

```
    epoch_acc = []

    Theta = Theta0
    for epoch in range(epochs):
        N = len(X)
        shuffle_idx = np.random.permutation(N)

        for start in range(0, N, batch_size):
            batch_idx = shuffle_idx[start:start+batch_size]
            grad = grad_l(Theta, X[batch_idx], Y[batch_idx])
            Theta -= lr * grad

        epoch_loss.append(l(Theta, X, Y))
        epoch_acc.append(accuracy(Theta, X, Y))
            # Update the loss_val list

    return Theta, epoch, epoch_loss, epoch_acc

X = np.vstack((class_0, class_1))
X = np.hstack((np.ones((X.shape[0], 1)), X))
Y = np.hstack((np.zeros(n_class_0_samples),
np.ones(n_class_1_samples)))

print("First 5 samples of X:\n", X[:5])
print("\n")
print("First 5 samples of Y:\n", Y[:5])

print("\n X shape:", X.shape, "Y shape:", Y.shape)
```

```
First 5 samples of X:
 [[ 1.         -1.03122138 -2.94913065]
 [ 1.         -1.75910219 -2.54883645]
 [ 1.         -1.61269596 -2.9710488 ]
 [ 1.         -2.10609275 -1.53565731]
 [ 1.         -1.99196496 -1.18601649]]


First 5 samples of Y:
 [0. 0. 0. 0. 0.]

 X shape: (200, 3) Y shape: (200,)
```

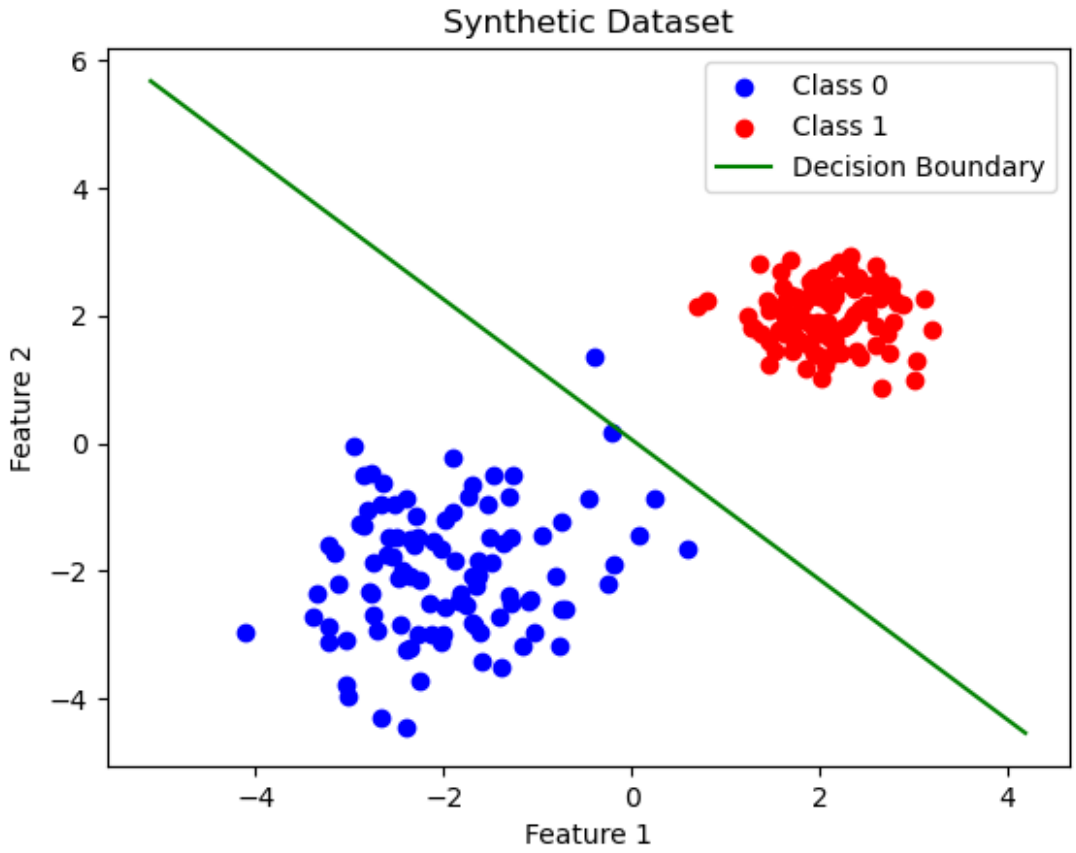
1. Train it using simple Gradient Descent on the full dataset. **Note:** the computation of $\nabla L(\Theta; X, Y)$ for this choice of ℓ is given in the teaching note.

```
Theta0 = np.zeros(X.shape[1])
batch_size = X.shape[0]
Theta, epoch, epoch_loss, epoch_acc = SGD(l, grad_l, X, Y,
Theta0=Theta0, lr=0.1, batch_size=batch_size, epochs=10)
```

1. Visualize the learned **decision boundary**:

- plot the line $\{\Theta^T x = 0\}$,
- overlay with the dataset.

```
plt.scatter(class_0[:, 0], class_0[:, 1], color='blue', label='Class
0')
plt.scatter(class_1[:, 0], class_1[:, 1], color='red', label='Class
1')

min_max_of_feature_1 = np.array([min(X[:,1])-1, max(X[:,1])+1])
feature_2 = -(Theta[0] + Theta[1] * min_max_of_feature_1) / Theta[2]
plt.plot(min_max_of_feature_1, feature_2, label='Decision Boundary',
color='green')

plt.title('Synthetic Dataset')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```



1. Comment on why the decision boundary is linear.

Exercise 2: SGD on Logistic Regression

Use the same synthetic dataset used in the previous exercise, but train logistic regression using **SGD** with the following choices:

1. Try different batch sizes:
   - $N_{\text{batch}}=1$,
   - $N_{\text{batch}}=10$,
   - $N_{\text{batch}}=N$ (full GD).

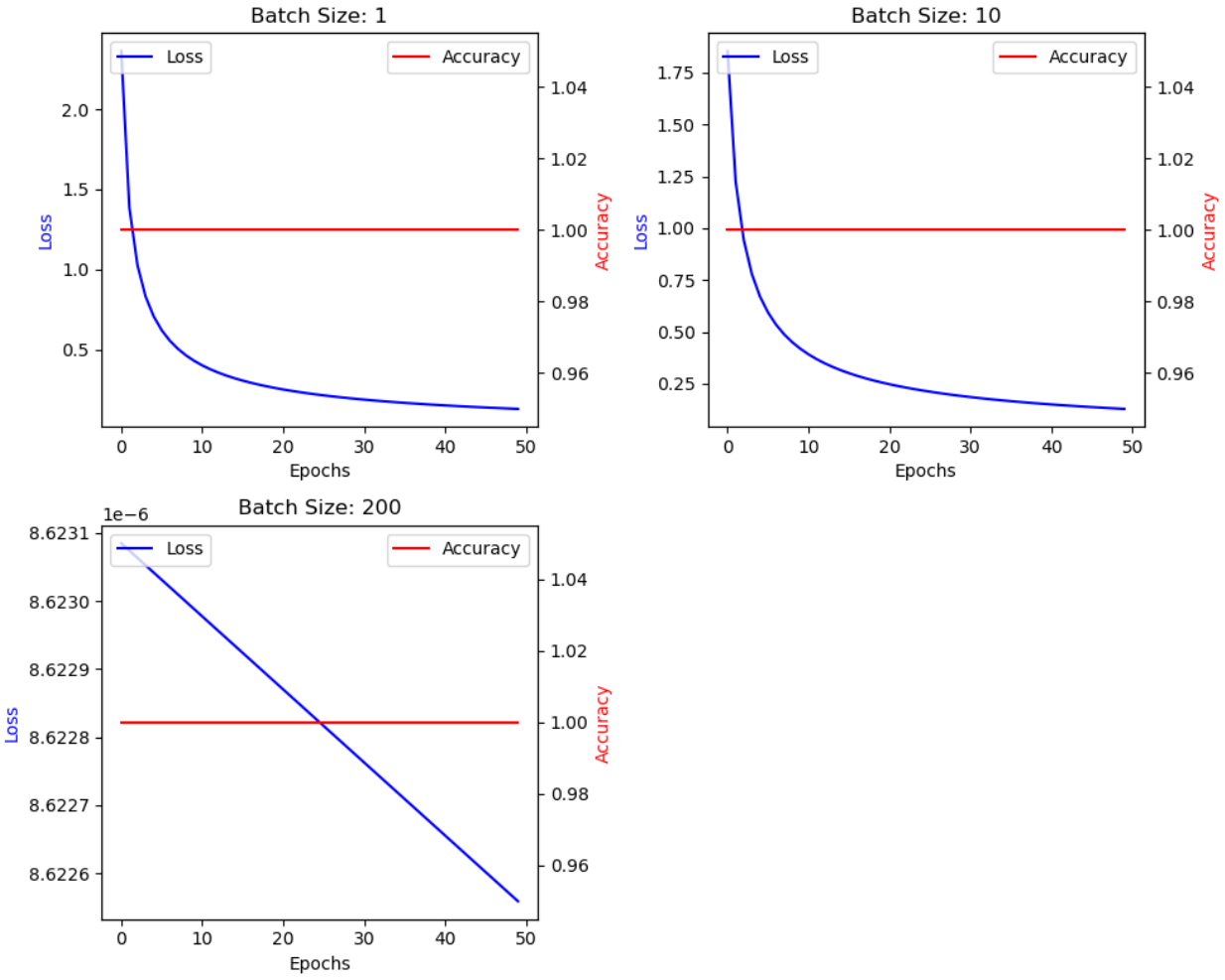
```
batch_sizes = [1, 10, X.shape[0]]

results = pd.DataFrame([],columns=['Batch Size', 'Theta', 'Final
Loss', 'Final Accuracy','Loss History', 'Accuracy History'])

for batch_size in batch_sizes:
    Theta0 = np.zeros(X.shape[1])
    Theta, epoch, epoch_loss, epoch_acc = SGD(l, grad_l, X, Y,
Theta0=Theta0, lr=0.1, batch_size=batch_size, epochs=50)
    results.loc[len(results)] = [batch_size, Theta, epoch_loss[-1],
epoch_acc[-1], epoch_loss, epoch_acc]


display(results.sort_values(by='Final Accuracy', ascending=False))
```

```
   Batch Size                                              Theta
Final Loss  \
0           1  [-0.8930001306206973, 2.930795967301779, 2.474...
0.129199
1          10  [-0.8957492503576683, 2.9294798916602773, 2.48...
0.128580
2         200  [-4.2251669504487595e-05, 21.090594498307944, ...
0.000009

   Final Accuracy                                        Loss
History  \
0             1.0  [2.3640703323637613, 1.3875034230363885, 1.027...

1             1.0  [1.855223760635221, 1.2248969128599845, 0.9434...

2             1.0  [8.623084685496744e-06, 8.623073951212868e-06,...


                                      Accuracy History
0  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, ...
1  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, ...
2  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, ...
```

1. For each setting:
   - Plot the loss vs epoch,

– Plot the classification accuracy vs epoch.

```
n_plots = len(batch_sizes)
n_cols = 2
n_rows = (n_plots + 1) // n_cols
fig, axes = plt.subplots(n_rows, n_cols, figsize=(5 * n_cols, 4 *
n_rows))
axes = axes.flatten()
for index, row in results.iterrows():
    batch_size = row['Batch Size']
    loss_history = row['Loss History']
    acc_history = row['Accuracy History']

    ax = axes[index]
    ax.plot(loss_history, label='Loss', color='blue')
    ax.set_xlabel('Epochs')
    ax.set_ylabel('Loss', color='blue')
    ax2 = ax.twinx()
    ax2.plot(acc_history, label='Accuracy', color='red')
    ax2.set_ylabel('Accuracy', color='red')
    ax.set_title(f'Batch Size: {batch_size}')
    ax.legend(loc='upper left')
    ax2.legend(loc='upper right')

# Hide any unused subplots
for j in range(index + 1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()
```



1. Compare the stability and speed of convergence over the choice of different batch sizes.
2. Why the gradients become noisier for small batches? Why larger batches give smoother curves?

Exercise 3: Evaluation Metrics on a Synthetic Dataset

Using the logistic regression model trained above:

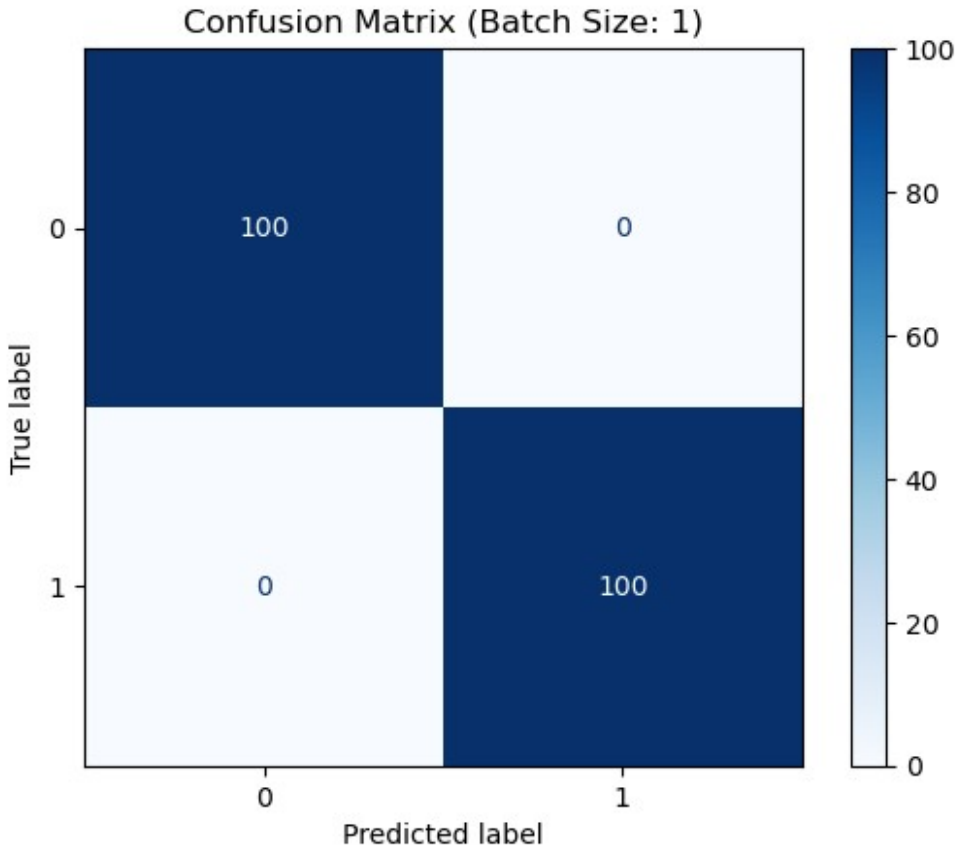
1. Compute predicted probabilities $\hat{y}_i = f_\Theta(x_i)$.
2. Convert them to binary predictions $\hat{y}_i \in \{0,1\}$ using threshold 0.5.
3. Compute:
   - Confusion matrix (TP, FP, FN, TN),
   - Accuracy,
   - Precision,
   - Recall,
   - F1-score.

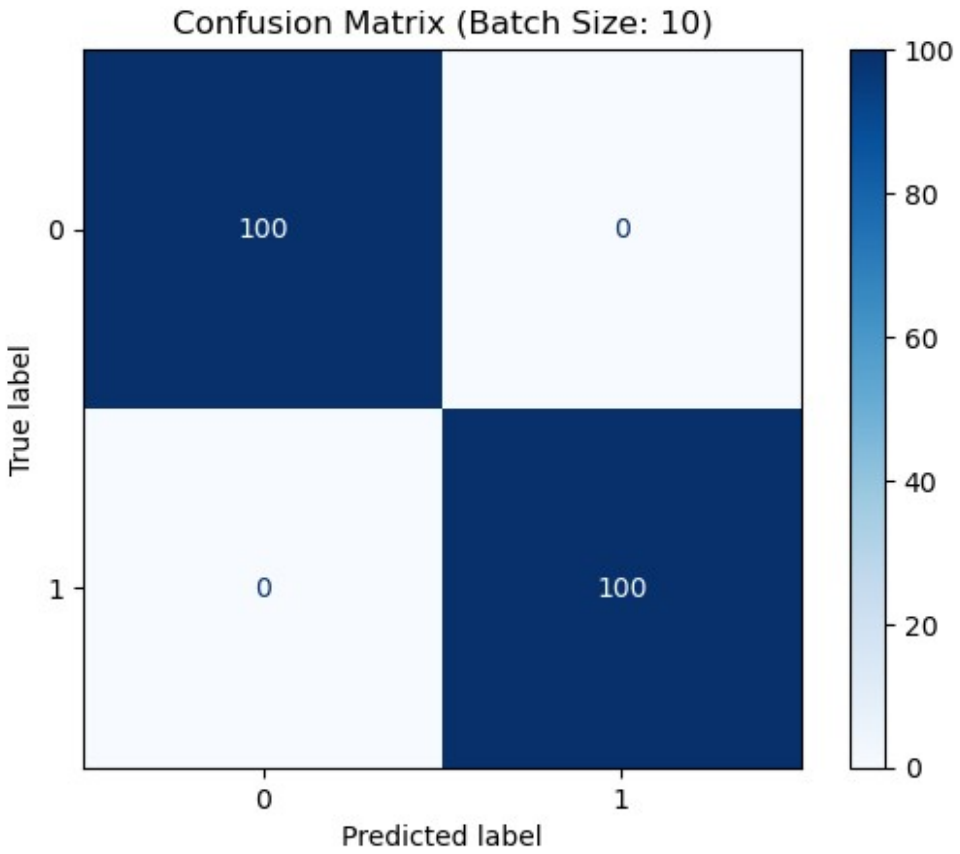
```
def classification_report(Y, Y_hat):
    TP = np.sum((Y == 1) & (Y_hat == 1))
    TN = np.sum((Y == 0) & (Y_hat == 0))
    FP = np.sum((Y == 0) & (Y_hat == 1))
    FN = np.sum((Y == 1) & (Y_hat == 0))

    accuracy = (TP + TN) / len(Y)
    precision = TP / (TP + FP) if (TP + FP) > 0 else 0
    recall = TP / (TP + FN) if (TP + FN) > 0 else 0
    f1_score = 2 * (precision * recall) / (precision + recall) if
(precision + recall) > 0 else 0

    confusion_matrix = np.array([[TP, FP],
                                 [FN, TN]])

    report = {
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1-Score': f1_score,
        'TP': TP,
        'TN': TN,
        'FP': FP,
        'FN': FN,
        'Confusion Matrix': confusion_matrix
    }
    return report

classification_reports = pd.DataFrame([], columns=['Batch Size',
'Accuracy', 'Precision', 'Recall', 'F1-
Score','TP','TN','FP','FN','Confusion Matrix'])
for index, row in results.iterrows():
    batch_size = row['Batch Size']
    Theta = row['Theta']
    Y_hat = (sigmoid(X @ Theta) >= 0.5).astype(int)
    report = classification_report(Y, Y_hat)
    classification_reports.loc[len(classification_reports)] =
[batch_size, report['Accuracy'], report['Precision'],
report['Recall'], report['F1-Score'], report['TP'], report['TN'],
report['FP'], report['FN'], report['Confusion Matrix']]

display(classification_reports)
```

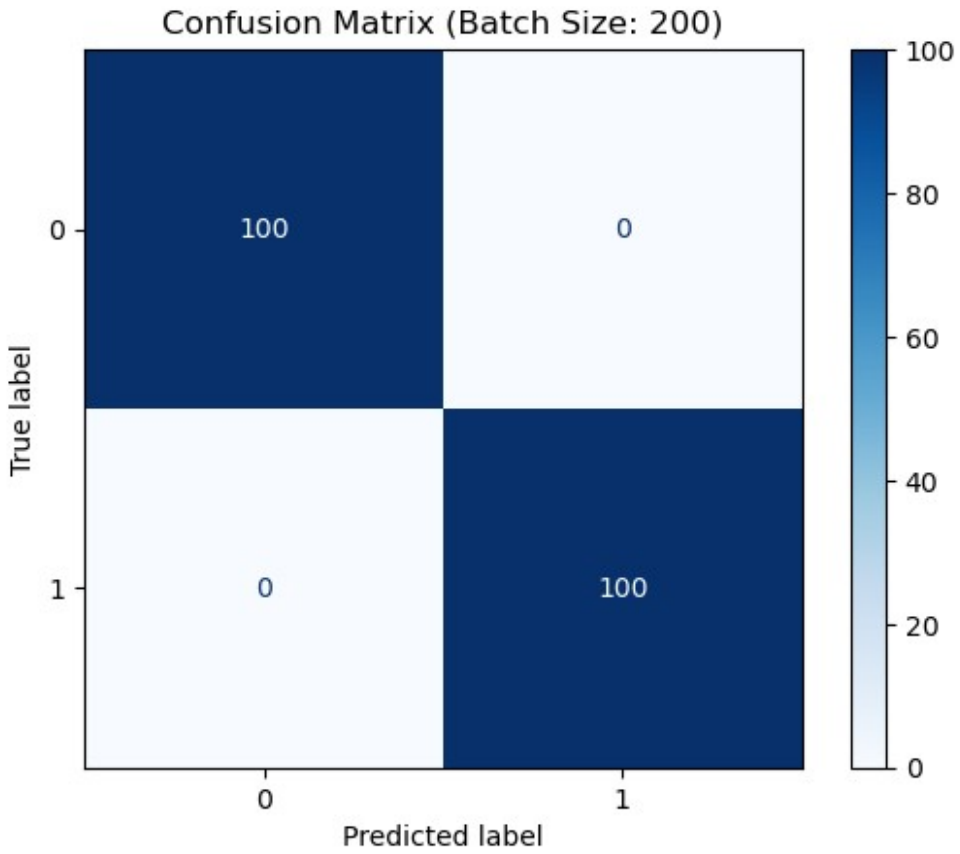
	Batch Size	Accuracy	Precision	Recall	F1-Score	TP	TN	FP	FN
0	1	1.0	1.0	1.0	1.0	100	100	0	0
1	10	1.0	1.0	1.0	1.0	100	100	0	0
2	200	1.0	1.0	1.0	1.0	100	100	0	0

```
       Confusion Matrix
0  [[100, 0], [0, 100]]
1  [[100, 0], [0, 100]]
2  [[100, 0], [0, 100]]
```

```
from sklearn.metrics import ConfusionMatrixDisplay

for index, row in classification_reports.iterrows():
    batch_size = row['Batch Size']
    cm = row['Confusion Matrix']
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=[0, 1])
    disp.plot(cmap=plt.cm.Blues)
    plt.title(f'Confusion Matrix (Batch Size: {batch_size})')
    plt.show()
```




Confusion Matrix (Batch Size: 10)
100
0
0
100
0
1
True label
0
1
Predicted label
100
80
60
40
20
0



1. Modify the threshold to:
   - 0.3,
   - 0.7, and repeat.

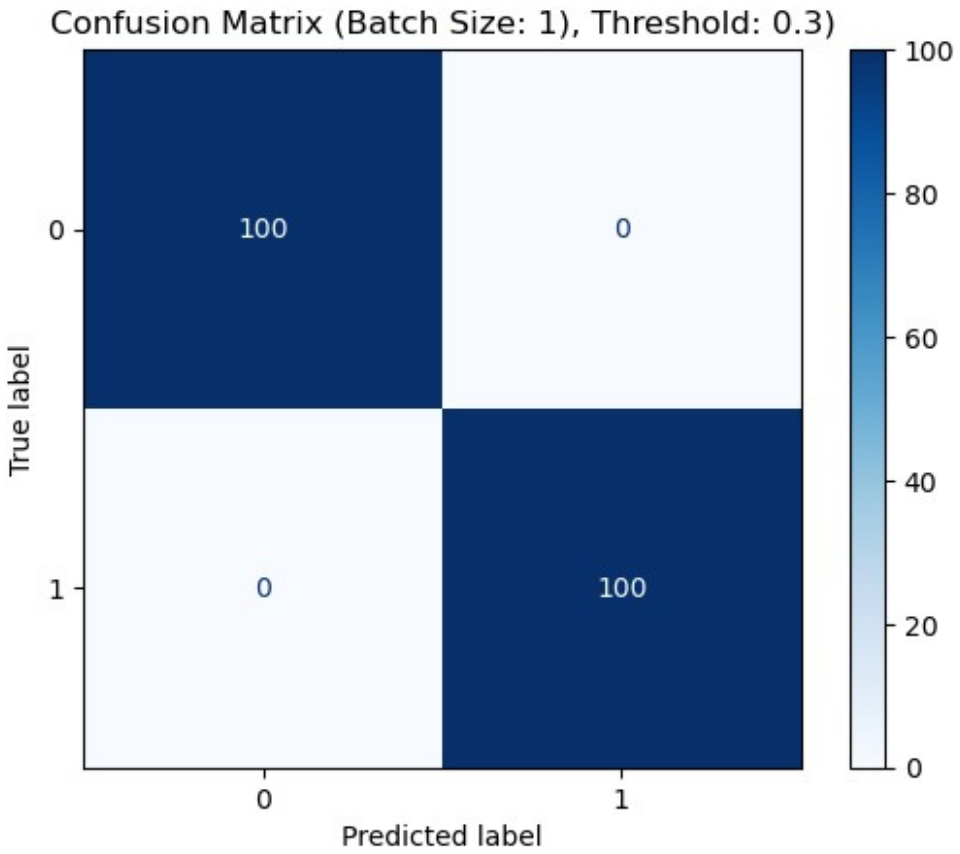
```
thresholds = [0.3, 0.7]
classification_reports = pd.DataFrame([], columns=['Threshold','Batch
Size', 'Accuracy', 'Precision', 'Recall', 'F1-
Score','TP','TN','FP','FN','Confusion Matrix'])
for threshold in thresholds:
    for index, row in results.iterrows():
        batch_size = row['Batch Size']
        Theta = row['Theta']
        Y_hat = (sigmoid(X @ Theta) >= threshold).astype(int)
        report = classification_report(Y, Y_hat)
        classification_reports.loc[len(classification_reports)] =
[threshold, batch_size, report['Accuracy'], report['Precision'],
report['Recall'], report['F1-Score'], report['TP'], report['TN'],
report['FP'], report['FN'], report['Confusion Matrix']]

display(classification_reports.sort_values(by=['Accuracy']))
```

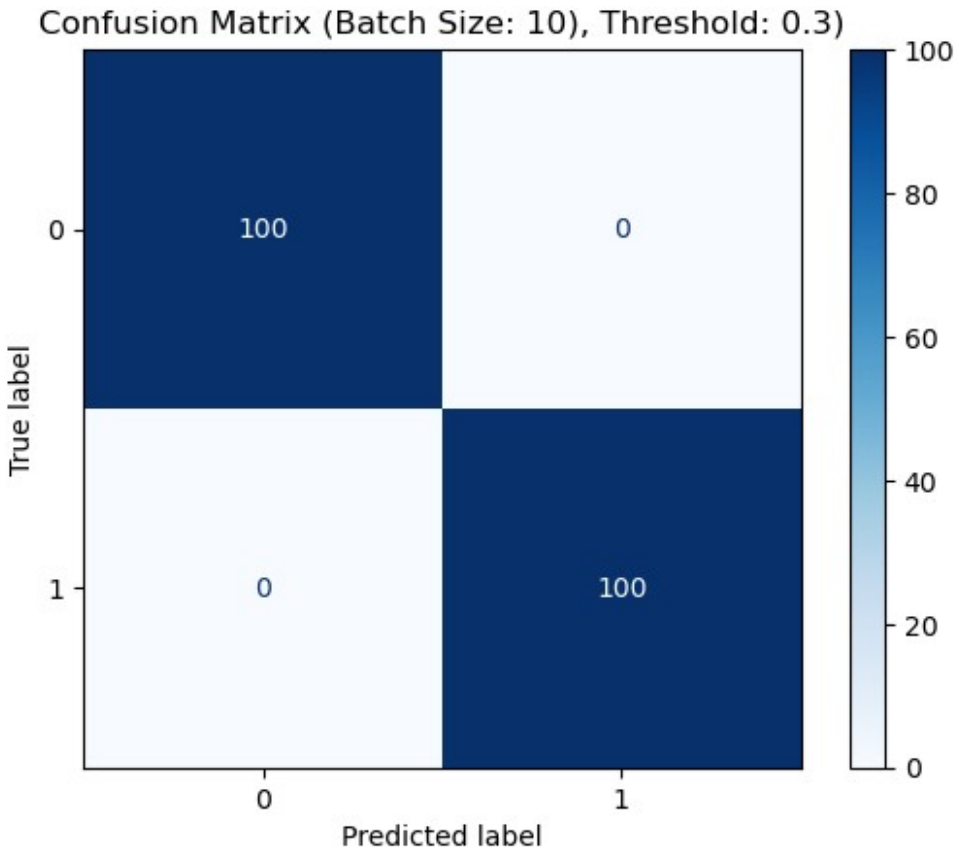
```
   Threshold  Batch Size  Accuracy  Precision  Recall  F1-Score   TP
TN  FP  \
0        0.3           1       1.0        1.0     1.0       1.0  100
```

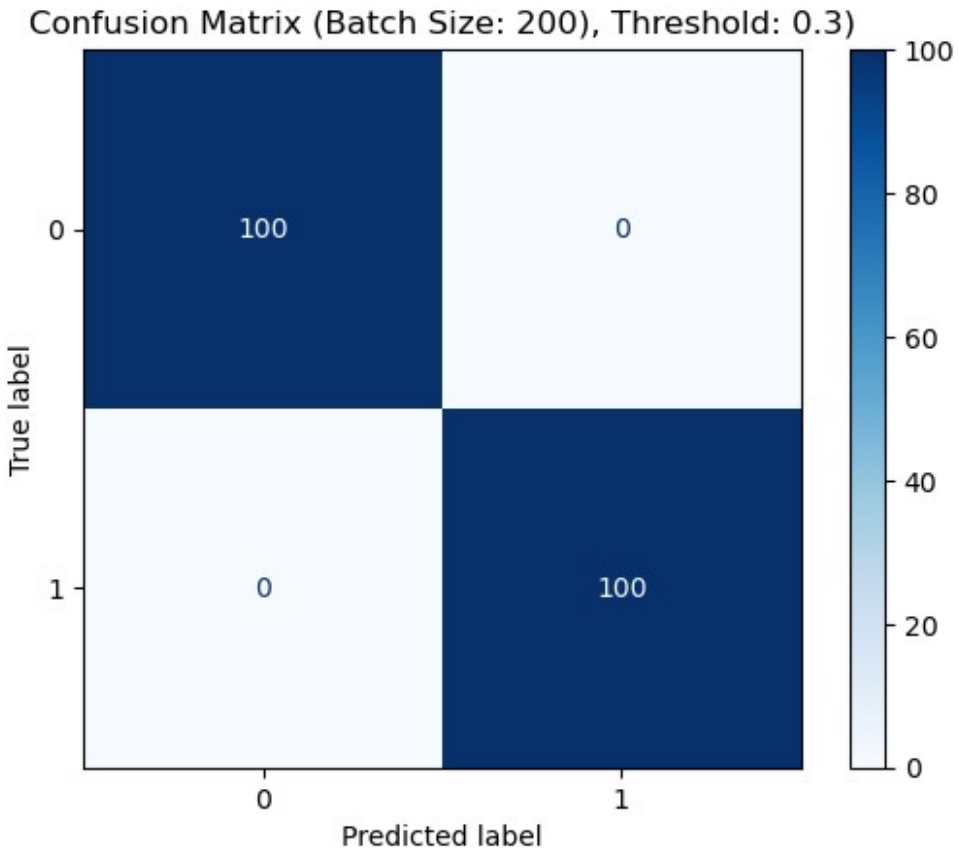
```
100   0
1         0.3           10        1.0         1.0      1.0         1.0  100
100   0
2         0.3          200        1.0         1.0      1.0         1.0  100
100   0
3         0.7            1        1.0         1.0      1.0         1.0  100
100   0
4         0.7           10        1.0         1.0      1.0         1.0  100
100   0
5         0.7          200        1.0         1.0      1.0         1.0  100
100   0
```

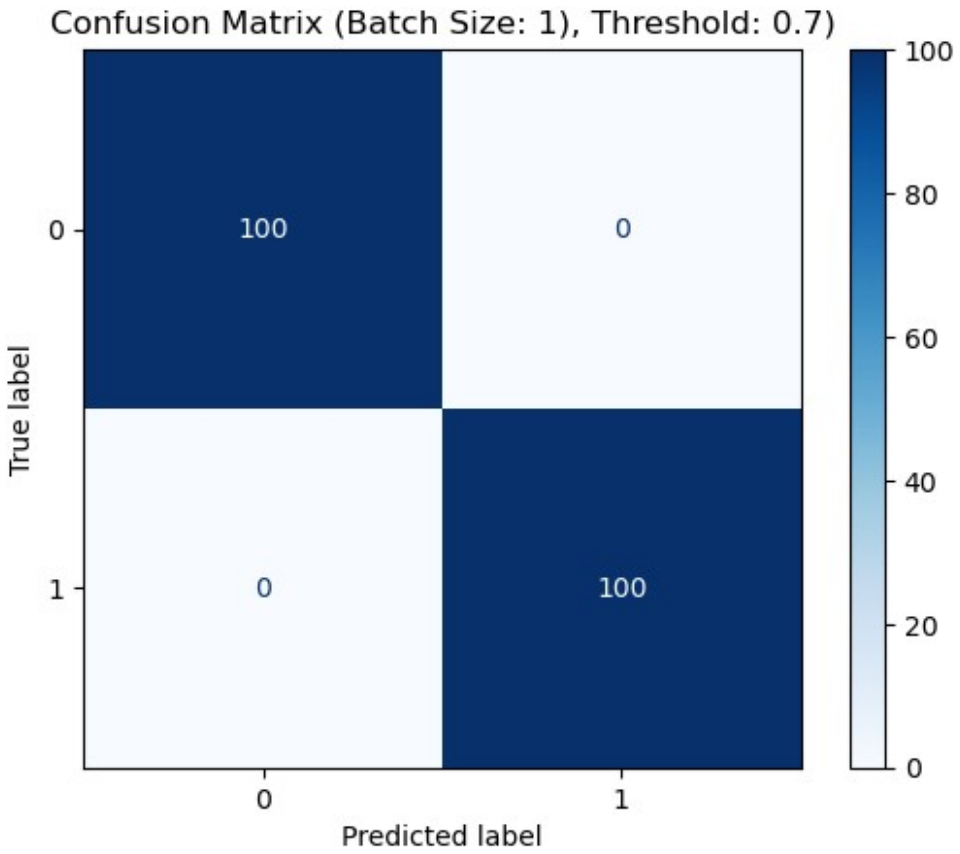
```
   FN      Confusion Matrix
0   0  [[100, 0], [0, 100]]
1   0  [[100, 0], [0, 100]]
2   0  [[100, 0], [0, 100]]
3   0  [[100, 0], [0, 100]]
4   0  [[100, 0], [0, 100]]
5   0  [[100, 0], [0, 100]]
```

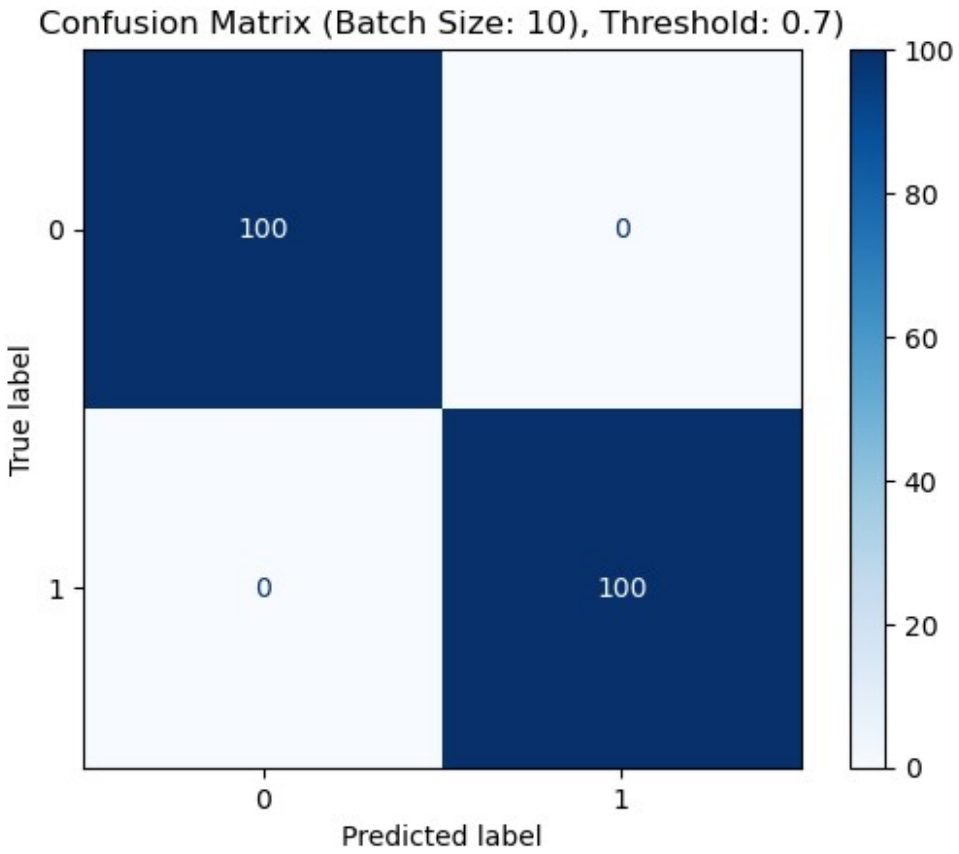
```
from sklearn.metrics import ConfusionMatrixDisplay

for index, row in classification_reports.iterrows():
    batch_size = row['Batch Size']
    cm = row['Confusion Matrix']
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=[0, 1])
    disp.plot(cmap=plt.cm.Blues)
    plt.title(f'Confusion Matrix (Batch Size: {batch_size}),
Threshold: {row["Threshold"]})')
    plt.show()
```

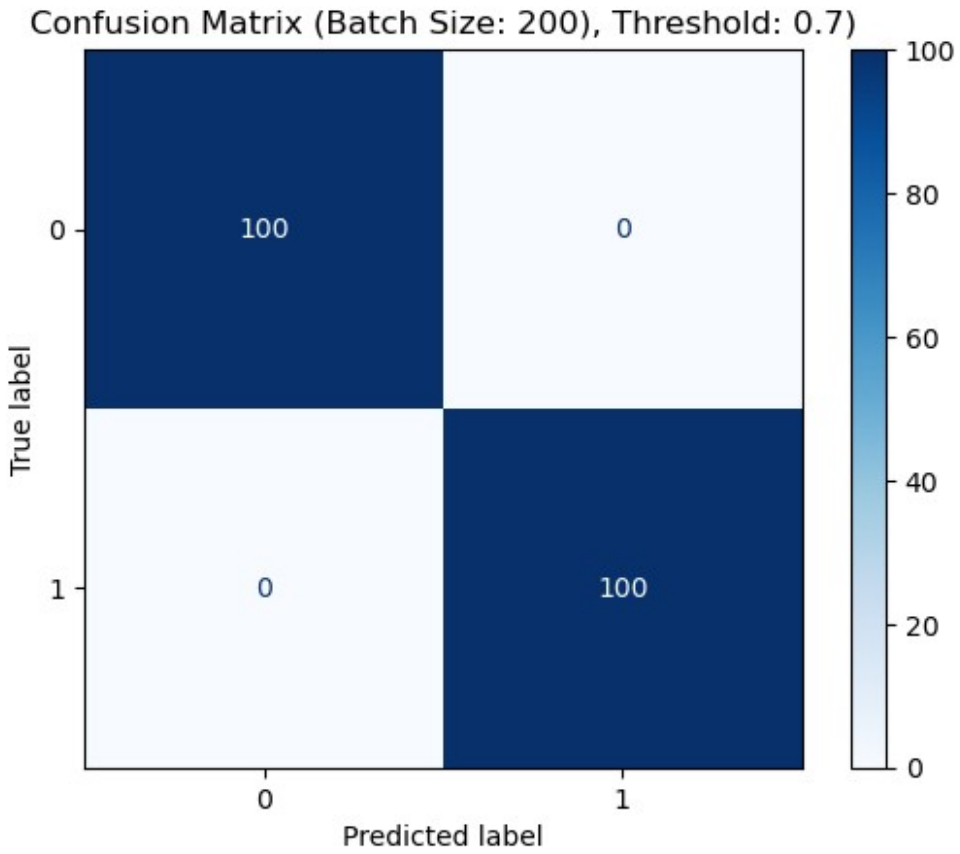

Confusion Matrix (Batch Size: 1), Threshold: 0.3)
100
0
0
100
0
1
True label
0
1
Predicted label
100
80
60
40
20
0


Confusion Matrix (Batch Size: 10), Threshold: 0.3)
100
0
0
100
0
1
True label
0
1
Predicted label
100
80
60
40
20
0


Confusion Matrix (Batch Size: 200), Threshold: 0.3)
100
0
0
100
True label
Predicted label
0
1
0
1
100
80
60
40
20
0


Confusion Matrix (Batch Size: 1), Threshold: 0.7)
100
0
0
100
0
1
True label
0
1
Predicted label
100
80
60
40
20
0


Confusion Matrix (Batch Size: 10), Threshold: 0.7)
100
0
0
100
True label
Predicted label
0
1
0
1
100
80
60
40
20
0



1. Comment on:
   - How lower thresholds increase recall and lower precision,
   - How higher thresholds increase precision and reduce recall,
   - Why classification metrics depend on the application (as discussed in class).

Exercise 4: Logistic Regression on a Real Dataset

Reproduce the pipeline from the book using the **Pima Indians Diabetes Dataset**, available at https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database.

This dataset contains medical measurements and a binary label indicating diabetes diagnosis.

1. Preprocess the data as done in class:
   - Download `diabetes.csv`.
   - Extract features X and labels y.
   - **Standardize the features** (mean 0, variance 1).
     Explain why normalization is required, connecting to:
     - conditioning,
     - stable optimization,
     - meaningful gradient magnitudes.
   - Add a bias column of 1s.

```
df0 = pd.read_csv('./diabetes.csv')
df0.shape
```

```
(768, 9)
```

```
df0.head()
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin
BMI  \
0            6      148             72             35        0  33.6

1            1       85             66             29        0  26.6

2            8      183             64              0        0  23.3

3            1       89             66             23       94  28.1

4            0      137             40             35      168  43.1


   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

```
X = df0.drop(columns=['Outcome']).values
Y = df0['Outcome'].values

X.shape, Y.shape
```

```
((768, 8), (768,))
```

```
X_mean, X_std = X.mean(axis=0), X.std(axis=0)
X = (X - X_mean) / X_std

X = np.hstack([np.ones((X.shape[0],1)), X])
```

1. Implement logistic regression from scratch (sigmoid + BCE + gradient), and optimize using **SGD** with:
   - a batch size of 32, a learning rate of 10^{-3}, and 200 epochs.
   - for each epoch, track full-dataset BCE loss and full-dataset accuracy.
   - in a plot, visualize the behavior of Loss vs epoch and Accuracy vs epoch. Comment the results.

```
def sigmoid(t):
    return 1 / (1 + np.exp(-t))

def l(Theta, X, Y):
    Y_hat = sigmoid(X @ Theta)
```

```
    return - np.sum(Y * np.log(Y_hat) + (1-Y) * np.log(1 - Y_hat))

def grad_l(Theta, X, Y):
    Y_hat = sigmoid(X @ Theta)
    return X.T @ (Y_hat - Y)

def accuracy(Theta, X, Y):
    preds = sigmoid(X @ Theta) >= 0.5
    return (preds == Y).mean()

def SGD(l, grad_l, X, Y, Theta0, lr=1e-3, batch_size=32, epochs=200):
    epoch_loss = []
    epoch_acc = []

    Theta = Theta0
    for epoch in range(epochs):
        N = len(X)
        shuffle_idx = np.random.permutation(N)

        for start in range(0, N, batch_size):
            batch_idx = shuffle_idx[start:start+batch_size]
            grad = grad_l(Theta, X[batch_idx], Y[batch_idx])
            Theta -= lr * grad

        epoch_loss.append(l(Theta, X, Y))
        epoch_acc.append(accuracy(Theta, X, Y))
            # Update the loss_val list

    return Theta, epoch, epoch_loss, epoch_acc
```

1. Train the same model using **Adam** (using the formulas from class):

$$\Theta_{k+1} = \Theta_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}.$$

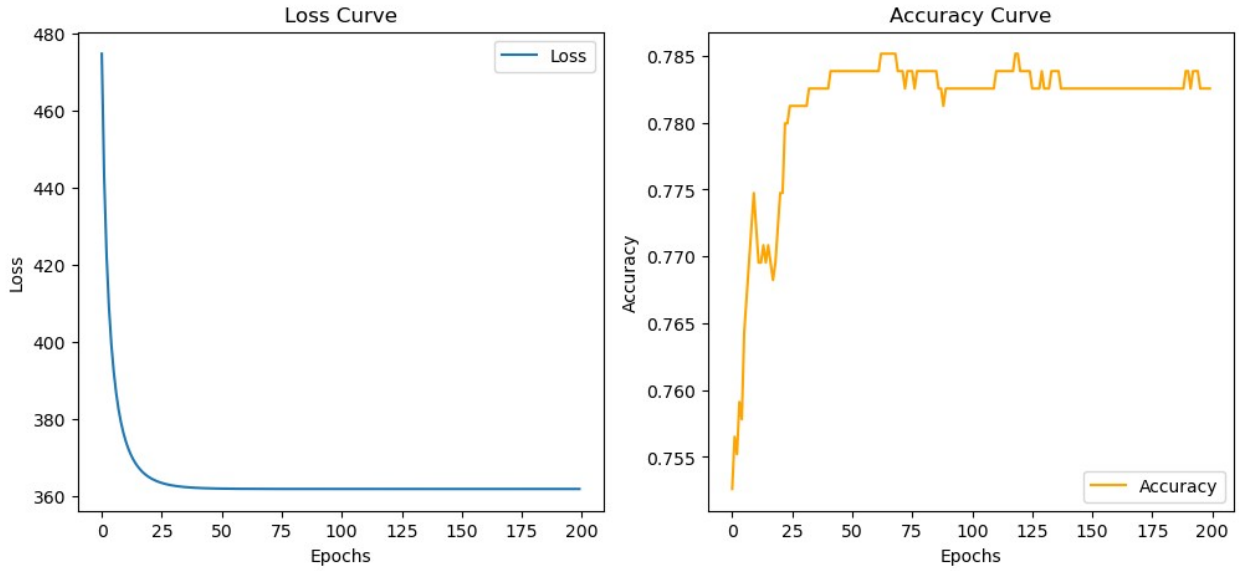
with a learning rate of 10^{-3}, a batch size of 32, and 200 epochs. Then, plot SGD vs Adam loss and accuracy curves.

```
Theta0 = np.zeros(X.shape[1])
Theta, epoch, epoch_loss, epoch_acc = SGD(l, grad_l, X, Y,
Theta0=Theta0)
```

1. For each method, evaluate:
   - Final accuracy,
   - Confusion matrix,
   - Precision, Recall, F1

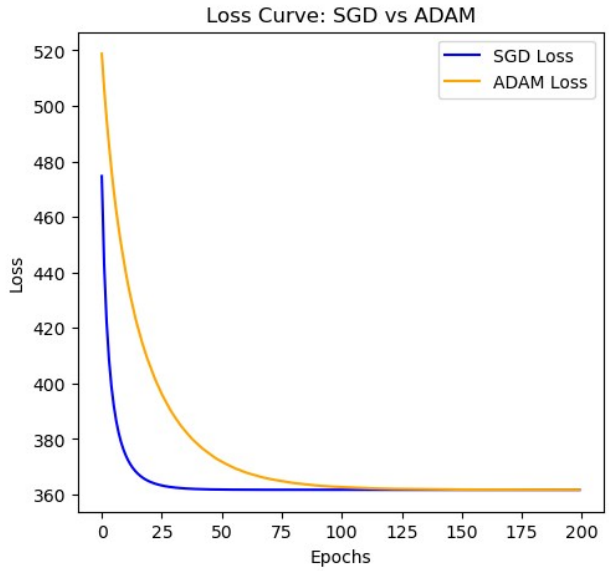
```
# plot loss and accuracy curves
plt.figure(figsize=(12, 5))
```

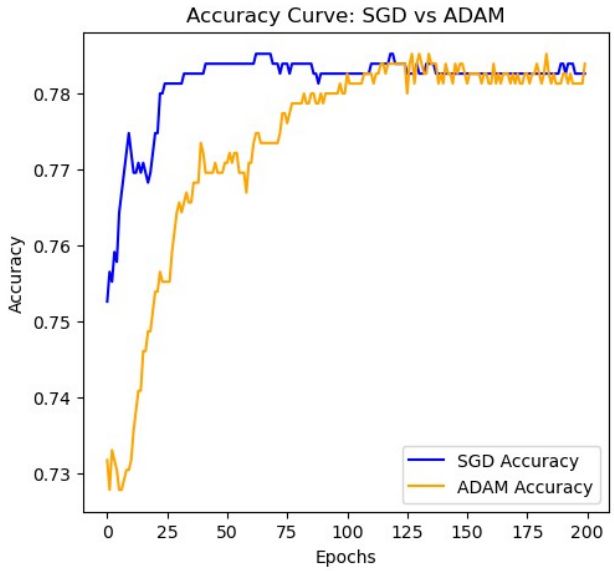
```
plt.subplot(1, 2, 1)
plt.plot(epoch_loss, label='Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Loss Curve')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(epoch_acc, label='Accuracy', color='orange')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Accuracy Curve')
plt.legend()
plt.show()
```




```
def ADAM(l, grad_l, X, Y, Theta0, lr=1e-3, batch_size=32, epochs=200,
beta1=0.9, beta2=0.999, eps=1e-8):
    epoch_loss = []
    epoch_acc = []

    Theta = Theta0

    m = np.zeros_like(Theta)
    v = np.zeros_like(Theta)
    t = 0

    for epoch in range(epochs):
        N = len(X)
        shuffle_idx = np.random.permutation(N)

        for start in range(0, N, batch_size):
```

```
            batch_idx = shuffle_idx[start:start+batch_size]
            grad = grad_l(Theta, X[batch_idx], Y[batch_idx])

            t += 1
            m = beta1 * m + (1 - beta1) * grad
            v = beta2 * v + (1 - beta2) * (grad * grad)
            m_hat = m / (1 - beta1**t)
            v_hat = v / (1 - beta2**t)
            Theta -= lr * (m_hat / (np.sqrt(v_hat) + eps))


        epoch_loss.append(l(Theta, X, Y))
        epoch_acc.append(accuracy(Theta, X, Y))
            # Update the loss_val list

    return Theta, epoch, epoch_loss, epoch_acc

Theta0 = np.zeros(X.shape[1])
adam_Theta, adam_epoch, adam_epoch_loss, adam_epoch_acc = ADAM(l,
grad_l, X, Y, Theta0=Theta0)

# Plot SDG vs ADAM loss curves
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(epoch_loss, label='SGD Loss', color='blue')
plt.plot(adam_epoch_loss, label='ADAM Loss', color='orange')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Loss Curve: SGD vs ADAM')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(epoch_acc, label='SGD Accuracy', color='blue')
plt.plot(adam_epoch_acc, label='ADAM Accuracy', color='orange')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Accuracy Curve: SGD vs ADAM')
plt.legend()
plt.show()
```

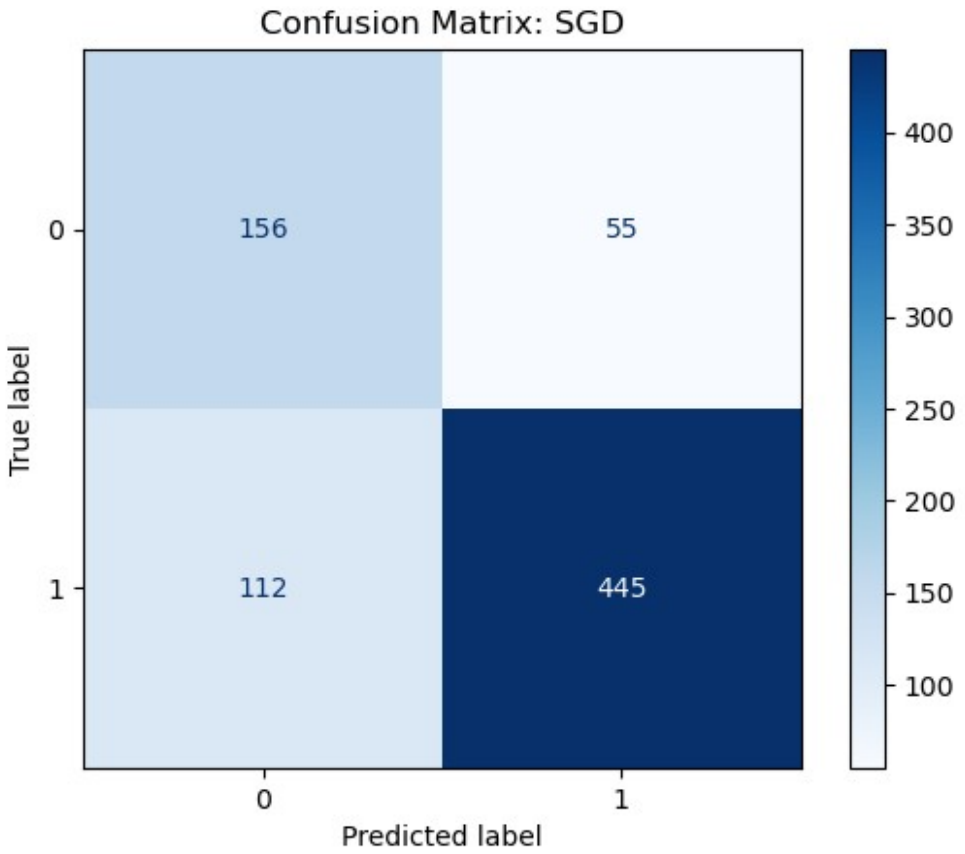


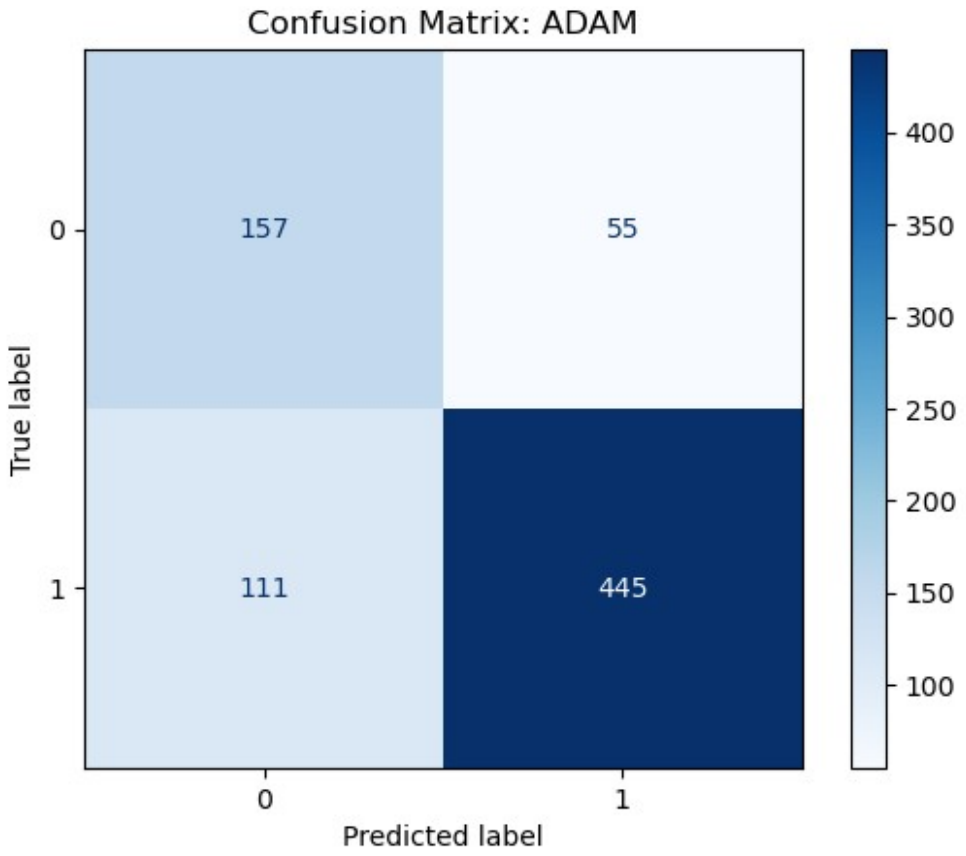



```
Y_hat_SGD = (sigmoid(X @ Theta) >= 0.5).astype(int)
Y_hat_ADAM = (sigmoid(X @ adam_Theta) >= 0.5).astype(int)
SGD_report = classification_report(Y, Y_hat_SGD)
ADAM_report = classification_report(Y, Y_hat_ADAM)

display(pd.DataFrame([SGD_report, ADAM_report], index=['SGD',
'ADAM']))
```

```
      Accuracy  Precision    Recall  F1-Score   TP   TN  FP   FN  \
SGD   0.782552   0.739336  0.582090  0.651357  156  445  55  112
ADAM  0.783854   0.740566  0.585821  0.654167  157  445  55  111

             Confusion Matrix
SGD   [[156, 55], [112, 445]]
ADAM  [[157, 55], [111, 445]]
```

```
from sklearn.metrics import ConfusionMatrixDisplay

disp_SGD =
ConfusionMatrixDisplay(confusion_matrix=SGD_report['Confusion
Matrix'], display_labels=[0, 1])
disp_SGD.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix: SGD')
plt.show()

disp_ADAM =
ConfusionMatrixDisplay(confusion_matrix=ADAM_report['Confusion
Matrix'], display_labels=[0, 1])
disp_ADAM.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix: ADAM')
plt.show()
```


Confusion Matrix: SGD
156
55
112
445
0
1
True label
0
1
Predicted label
400
350
300
250
200
150
100



1. Discuss on which method converge faster, which oscillate more, and how this relates to adaptive learning rates discussed in class.

Optional Extension: From Logistic Regression to a Simple Neural Network

This optional section mirrors the final part of the supervised learning chapter.

Implement a neural network with:

- One hidden layer,
- ReLU activation,
- Sigmoid output layer.

You may reuse the implementation shown in the main text. Then:

1. Train the neural network on the **same** Kaggle dataset.
2. Track:
   - BCE loss vs epoch,
   - Accuracy vs epoch.
3. Compare and discuss the performance against logistic regression.

```
import numpy as np

def sigmoid(z):
    return np.where(z >= 0,
             1/(1+np.exp(-z)),
             np.exp(z)/(1+np.exp(z))  # prevent overflow when z is
large negative
            )

def relu(z):
    return np.maximum(0, z)

def relu_grad(z):
    g = np.zeros_like(z)
    g[z > 0] = 1.0
    return g

def bce_loss(y_hat, y, eps=1e-10):
    y_hat = np.clip(y_hat, eps, 1 - eps)
    return -(y*np.log(y_hat) + (1-y)*np.log(1 - y_hat)).mean()

def train_mlp_adam(X, y, hidden=16, lr=1e-2, epochs=200,
batch_size=32):
    N, d = X.shape

    W1 = np.random.normal(0, np.sqrt(2/d), size=(d, hidden))
    b1 = np.zeros((1, hidden))
    W2 = np.random.normal(0, np.sqrt(2/hidden), size=(hidden, 1))
    b2 = np.zeros((1, 1))

    losses, accs = [], []

    for _ in range(epochs):
        # Shuffle data
        idx = np.arange(len(X))
        np.random.shuffle(idx)

        X = X[idx, :]
        y = y[idx]

        for batch in range(0, N, batch_size):
            Xb, yb = X[batch:batch+batch_size],
y[batch:batch+batch_size]

            # forward
            z1 = Xb @ W1 + b1
            a1 = relu(z1)
            z2 = a1 @ W2 + b2
            y_hat = sigmoid(z2)
```

```
            # backward
            delta2 = (y_hat - yb) / len(yb)
            dW2 = a1.T @ delta2
            db2 = delta2.sum(axis=0, keepdims=True)

            delta1 = (delta2 @ W2.T) * relu_grad(z1)
            dW1 = Xb.T @ delta1
            db1 = delta1.sum(axis=0, keepdims=True)

            # update
            W1 -= lr * dW1
            b1 -= lr * db1
            W2 -= lr * dW2
            b2 -= lr * db2

        # track
        z1 = X @ W1 + b1
        a1 = relu(z1)
        z2 = a1 @ W2 + b2
        yhat_full = sigmoid(z2)

        losses.append(bce_loss(yhat_full, y))
        preds = (yhat_full >= 0.5).astype(int)
        accs.append((preds == y).mean())

    return (W1, b1, W2, b2), np.array(losses), np.array(accs)

Y= Y.reshape(-1,1)

X.shape, Y.shape
```

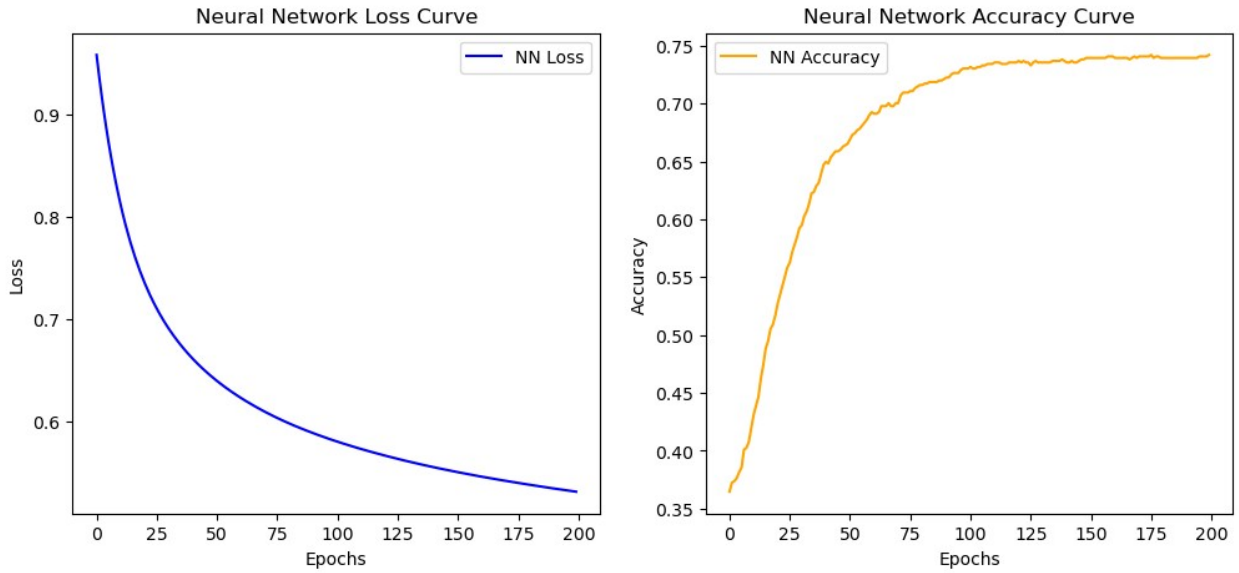
```
((768, 9), (768, 1))
```

```
theta_nn_sgd, loss_nn_sgd, acc_nn_sgd = train_mlp_adam(
    X, Y,
    hidden=16,
    lr=1e-3,
    epochs=200,
    batch_size=32,
)
print(f"Neural Network final accuracy: {acc_nn_sgd[-1]*100:.2f}%")
```

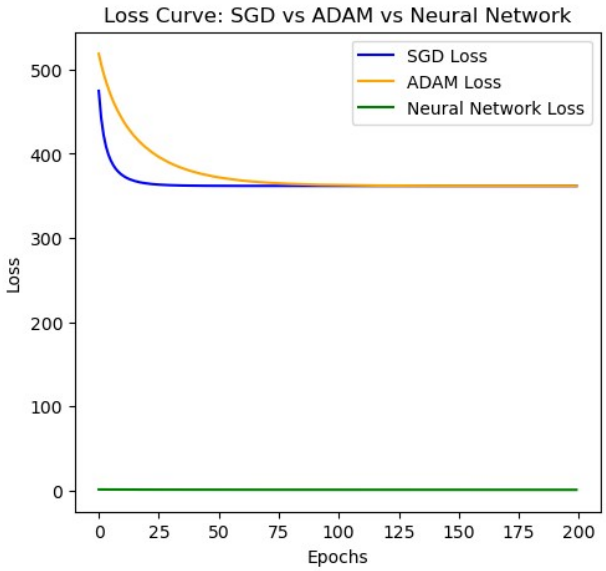
```
Neural Network final accuracy: 74.22%
```

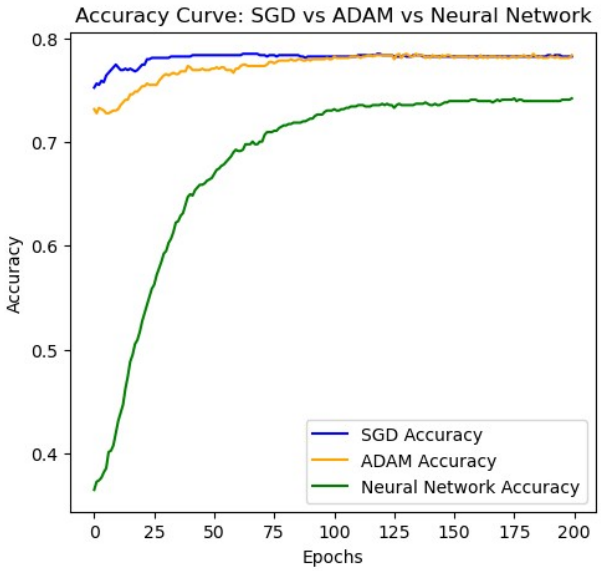
```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(loss_nn_sgd, label='NN Loss', color='blue')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Neural Network Loss Curve')
plt.legend()
```

```
plt.subplot(1, 2, 2)
plt.plot(acc_nn_sgd, label='NN Accuracy', color='orange')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Neural Network Accuracy Curve')
plt.legend()
plt.show()
```




```
# Plot loss and accuracy curves for Adam Vs SGD VS Neural Network
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(epoch_loss, label='SGD Loss', color='blue')
plt.plot(adam_epoch_loss, label='ADAM Loss', color='orange')
plt.plot(loss_nn_sgd, label='Neural Network Loss', color='green')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Loss Curve: SGD vs ADAM vs Neural Network')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(epoch_acc, label='SGD Accuracy', color='blue')
plt.plot(adam_epoch_acc, label='ADAM Accuracy', color='orange')
plt.plot(acc_nn_sgd, label='Neural Network Accuracy', color='green')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Accuracy Curve: SGD vs ADAM vs Neural Network')
plt.legend()

plt.show()
```


Loss Curve: SGD vs ADAM vs Neural Network
SGD Loss
ADAM Loss
Neural Network Loss
Loss
Epochs
0
100
200
300
400
500
0
25
50
75
100
125
150
175
200


Accuracy Curve: SGD vs ADAM vs Neural Network
SGD Accuracy
ADAM Accuracy
Neural Network Accuracy
Accuracy
Epochs
0.4
0.5
0.6
0.7
0.8
0
25
50
75
100
125
150
175
200